

Financial News Sentiment & Stock Price Analysis

A Production-Grade Data Science Portfolio Project

Author: Daniel Mituku

Project: Week 1 Stock Challenge (Enhanced Week 12)

Date: February 17, 2026

Repository: <https://github.com/Danielmituku/stock-challenge-week1>

Executive Summary

This project analyzes the relationship between financial news sentiment and stock price movements for six major tech stocks. Originally developed in Week 1, it has been transformed in Week 12 into a production-grade portfolio piece demonstrating software engineering best practices valued by finance sector employers.

Key Achievements

Metric	Week 1	Week 12	Improvement
Code Structure	1 file (398 lin	8 modules (1,20	3x more organiz
Test Coverage	0 tests	35 tests	improvement
Type Safety	0%	100% functions	Full type hints
Documentation	Basic README	Professional do	Complete overha
Visualization	Static notebook	Interactive das	User-accessible

1. Business Problem

The Challenge

Can we predict stock market movements by analyzing the sentiment of financial news?

Financial institutions spend billions on market research. Understanding the relationship between news sentiment and price movements could provide:

- Early warning signals for price changes

Stocks Analyzed

Ticker	Company	Sector
AAPL	Apple Inc.	Technology
AMZN	Amazon.com	E-commerce/Clou
GOOG	Alphabet Inc.	Technology
META	Meta Platforms	Social Media

MSFT	Microsoft Corp.	Technology
NVDA	NVIDIA Corp.	Semiconductors

2. Solution Architecture

System Design

DATA LAYER

News Data (1.4M headlines)	Stock Data (6 tickers)
raw_analyst_ratings.csv	AAPL, AMZN, GOOG, META,
- headline, date, publisher	MSFT, NVDA historical data
- stock ticker, URL	- OHLCV daily prices

PROCESSING LAYER (src/)

src/data/	src/analysis/	src/visualization/
loader.py	sentiment.py	plotting.py
- load_news_data()	- TextBlob	
- load_stock_data()	- VADER	
preprocessor.py	technical.py	
- clean_data()	- RSI, MACD, SMA	
- check_quality()	statistics.py	
	- correlation	

PRESENTATION LAYER

Streamlit Dashboard (app/dashboard.py)	
Price Chart Tab	- Candlestick + indicators
Returns Tab	- Distribution + cumulative
Sentiment Tab	- Time series + pie chart
Data Table Tab	- Raw data exploration

3. Technical Implementation (Week 12 Improvements)

3.1 Code Refactoring

Before (Week 1): Single utility file with no type hints

```
# notebooks/utis.py (OLD - no type hints, mixed concerns)
def load_data(file_path='../data/raw.csv', sample_size=None):
    print("Loading dataset...")
    if sample_size:
        df = pd.read_csv(file_path, nrows=sample_size)
    else:
        df = pd.read_csv(file_path)
    return df
```

After (Week 12): Modular package with full type hints and dataclasses

```
# src/data/loader.py (NEW - typed, documented, validated)
from typing import Optional, Union
from pathlib import Path
import pandas as pd
from ..config import DataConfig

def load_news_data(
    file_path: Optional[Union[str, Path]] = None,
    sample_size: Optional[int] = None,
    config: Optional[DataConfig] = None,
) -> pd.DataFrame:
    """
    Load the raw analyst ratings (news) dataset.

    Parameters
    -----
    file_path : str or Path, optional
        Path to the CSV file.
    sample_size : int, optional
        Number of rows to sample.
    config : DataConfig, optional
        Configuration object.

    Returns
    -----
    pd.DataFrame
        News data with columns: headline, date, publisher, stock

    Raises
    -----
    FileNotFoundError
        If the data file does not exist.
    """
    if config is not None:
        file_path = config.file_path
        sample_size = config.sample_size

    file_path = Path(file_path)
```

```

if not file_path.exists():
    raise FileNotFoundError(f"Data file not found: {file_path}")

# ... implementation

```

Configuration Dataclass:

```

# src/config.py
from dataclasses import dataclass, field
from typing import List, Optional

@dataclass
class TechnicalAnalysisConfig:
    """Configuration for technical analysis calculations."""

    rsi_period: int = 14
    rsi_overbought: float = 70.0
    rsi_oversold: float = 30.0
    macd_fast: int = 12
    macd_slow: int = 26
    macd_signal: int = 9
    sma_periods: List[int] = field(default_factory=lambda: [20, 50, 200])
    ema_periods: List[int] = field(default_factory=lambda: [12, 26])
    bollinger_period: int = 20
    bollinger_std: float = 2.0

```

3.2 Unit Testing Results

Test Coverage Summary:

Module	Tests	Pass Rate	Coverage
`src/config.py`	12	100%	95%
`src/analysis/s`	10	100%	88%
`src/analysis/t`	5	100%	92%
`src/analysis/s`	8	100%	90%
Total	**35**	**100%**	**91%**

Sample Test Output:

```

===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2
collected 35 items

tests/test_analysis.py::TestSentimentAnalysis::test_textblob_positive PASSED
tests/test_analysis.py::TestSentimentAnalysis::test_vader_negative PASSED
tests/test_analysis.py::TestTechnicalAnalysis::test_calculate_rsi PASSED
tests/test_analysis.py::TestTechnicalAnalysis::test_calculate_macd PASSED
tests/test_analysis.py::TestStatistics::test_gini_coefficient PASSED
tests/test_analysis.py::TestStatistics::test_correlation PASSED
tests/test_config.py::TestDataConfig::test_default_values PASSED

```

```
tests/test_config.py::TestDataConfig::test_invalid_config PASSED
... (27 more tests)
```

```
===== 35 passed in 13.91s =====
```

3.3 Interactive Dashboard

Dashboard Screenshot Descriptions:

Tab 1: Price Chart with Technical Indicators

AAPL Stock Price with Technical Indicators

[Candlestick Chart]

- Green/Red candles showing OHLC prices
- Orange line: SMA 20-day
- Blue line: SMA 50-day
- Red line: SMA 200-day
- Gray band: Bollinger Bands (+/-2)

[RSI Chart - 0 to 100 scale]

- Purple line: RSI value
- Red dashed: Overbought (70)
- Green dashed: Oversold (30)

[MACD Chart]

- Blue line: MACD
- Orange line: Signal
- Green/Red bars: Histogram

Tab 2: Returns Distribution

Daily Returns Distribution

[Histogram]

- Bell curve shape
- Mean: 0.13% daily
- Std: 1.80%
- Range: -10% to +12%

Cumulative Returns

[Line Chart]

- Starting at 1.0
- Ending at ~3.5x (250%)
- Shows growth over time

Tab 3: Sentiment Analysis

Sentiment Over Time

[Scatter Plot]

- X: Date
- Y: Sentiment (-1 to 1)

Category Distribution

[Pie Chart]

- Green: Positive (29%)
- Red: Negative (19%)

- Color: Red/Yellow/Green
- Gray: Neutral (52%)
- Horizontal line at 0

Key Metrics Panel:

Latest Price	Daily Change	Volatility	RSI	Period Return
\$185.92	+1.23%	1.80%	58.4 Neutral	+156.2%

4. Key Results and Findings

4.1 Week 1 Analysis Results

Dataset Statistics:

Metric	Value
Total Headlines	1,407,328
Date Range	2011-2020
Unique Publishe	3,000+
Average Headlin	73 characters
Valid Date Reco	3.98%

Sentiment Distribution (55,230 analyzed headlines):

Sentiment Category Breakdown:

Neutral: 52.13%

Positive: 28.66%

Negative: 19.20%

Sentiment by Stock:

Stock	Mean Sentiment	Category
AMZN	+0.190	Most Positive
NVDA	+0.181	Very Positive
AAPL	+0.177	Very Positive
MSFT	+0.125	Positive
META	+0.098	Slightly Positi
GOOG	+0.017	Nearly Neutral

Technical Indicator Results:

Stock	Mean Daily Retu	Volatility	Sharpe Ratio	Max Drawdown
NVDA	0.19%	2.89%	1.82	-18.76%
AAPL	0.13%	1.80%	1.45	-12.86%
AMZN	0.13%	2.18%	1.21	-14.05%
META	0.11%	2.53%	0.89	-26.39%

MSFT	0.10%	1.69%	1.18	-14.74%
GOOG	0.09%	1.73%	1.04	-11.10%

Correlation Analysis Limitation:

- Only 0.05-0.13% of trading days had matching news data

4.2 Week 12 Improvement Metrics

Code Quality Improvements:

Metric	Before	After	Improvement
Number of modul	1	8	8x more modular
Lines of code	398	1,247	Better organiza
Functions with	0	45	100% coverage
Documented func	12	45	275% increase
Cyclomatic comp	N/A	3.2	Low complexity

Testing Improvements:

Metric	Before	After
Unit tests	0	35
Test pass rate	N/A	100%
Code coverage	0%	91%
Test execution	N/A	13.9s

Dashboard Capabilities Added:

Feature	Description	Business Value
Stock Selection	6 tickers avail	Flexible analys
Date Range Filt	Custom time win	Historical comp
Technical Indic	RSI, MACD, SMA,	Trading signals
Sentiment View	Time series + d	News impact ana
Data Export	Raw data table	Further analysi

5. Business Impact Assessment

Value for Finance Sector

1. Risk Reduction Through Testing

- 35 unit tests ensure calculation accuracy

Quantified Portfolio Value

Skill Demonstra	Evidence	Finance Relevan
Software Engine	8 modular packa	Production-read
Quality Assuran	35 tests, 91% c	Reliable calcul

Data Visualizat	Interactive das	Stakeholder com
Documentation	Professional RE	Team collaborat
Python Proficie	Type hints, dat	Modern best pra

6. Limitations and Honest Assessment

Data Alignment Challenge

The correlation analysis faced a critical limitation:

News Data Coverage: || 3.98% valid dates

Stock Data Coverage: || 100% trading days

Overlap for Analysis: || 0.05-0.13%

Impact: Insufficient sample sizes (2-5 observations per stock) prevented statistically significant correlation conclusions.

Lesson Learned: Data preprocessing and alignment must be verified BEFORE conducting correlation analysis. This is a common real-world data science challenge.

What This Project Does NOT Prove

1. [X] Does NOT prove news sentiment predicts stock prices

What This Project DOES Demonstrate

1. [DONE] End-to-end data science workflow
-

7. Future Improvements

If Given More Time

Priority	Improvement	Expected Impact
1	Obtain news dat	Enable proper c
2	Add SHAP explai	Increase transp
3	Deploy dashboar	Public accessib
4	Add real-time d	Live analysis c
5	Extend to more	Broader market

Recommended Next Steps

Phase 1: Data Improvement

Obtain news data with timestamps matching trading days

Phase 2: Model Development

Train ML models using sentiment as features

Add SHAP visualizations for model explainability

Phase 3: Production Deployment

Deploy to cloud platform

Add authentication and user management

Implement real-time data streaming

8. How to Run This Project

Quick Start

```
# Clone repository
git clone https://github.com/Danielmituku/stock-challenge-week1.git
cd stock-challenge-week1

# Create virtual environment
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Run tests
pytest tests/ -v

# Launch dashboard
streamlit run app/dashboard.py
```

Project Structure

```
stock-challenge-week1/
├── app/
│   └── dashboard.py          # Streamlit dashboard
├── src/
│   ├── config.py            # Configuration dataclasses
│   ├── data/                # Data loading/preprocessing
│   ├── analysis/            # Sentiment, technical, statistics
│   └── visualization/       # Plotting utilities
├── tests/
│   ├── test_config.py       # 12 tests
│   └── test_analysis.py     # 23 tests
```

notebooks/	# Analysis notebooks
INTERIM_REPORT.md	# Progress report
FINAL_REPORT.md	# This document
README.md	# Project documentation

9. Conclusion

This project demonstrates the journey from initial data analysis to a production-ready portfolio piece. Key accomplishments:

Technical Excellence

- 8 modular packages with full type hints

Professional Standards

- Clean, documented, maintainable code

Finance Sector Readiness

- Reliable calculations verified by tests

Honest Assessment

- Data limitations prevented correlation conclusions

Repository: <https://github.com/Danielmituku/stock-challenge-week1>

Dashboard: Run `streamlit run app/dashboard.py`

Tests: Run `pytest tests/ -v` (35 tests, all passing)

This report demonstrates production-grade data science practices including modular code design, comprehensive testing, interactive visualization, and honest communication of limitations-key competencies valued by finance sector employers.